Canon X-07 – Applications en assembleur

Synthèse du livre

Applications en assembleur dans l'univers du Canon X-07

A. Tonic, E. Arevian, P. Millet, J.-J. Rousseau – Éditions Neptune, 1986

Ce livre fait suite aux *Mystères du X-07*, dont il reprend les adresses et les routines. Il comporte trois parties : « Soft » (utiliser des routines en langage machine depuis le BASIC), « Hard » (interruptions, entrées/sorties, interface vidéo X-720) et « Logiciel » (douze utilitaires et jeux). Ce document reprend l'ensemble sous forme de tableaux et de listings. Les numéros de page (p. xx) renvoient au livre. Sauf mention contraire, les valeurs sont en hexadécimal.
Les programmes courts ont été vérifiés en désassemblant les octets de leurs lignes `DATA`. Les coquilles et les bogues relevés sont signalés par **N.d.R.** . Les longs vidages hexadécimaux des logiciels de la 3e partie ne sont pas recopiés : leur transcription depuis le scan serait trop peu fiable. On renvoie aux pages du livre.

Table des matières

Présentation du livre **2**

1re partie : Soft **2**

1 Utilisation de routines en langage machine (p. 15-25) **2**

1.1 Où implanter la routine ? (p. 15-19) . . . 2

1.2 Appel et passage d'arguments (p. 19-21) . . . 3

1.3 Exemples (p. 22-25) . . . 4

2 Codage des variables (p. 26-30) **5**

2.1 Variables simples . . . 5

2.2 Tableaux . . . 5

3 Routines de la ROM (p. 31-40) **6**

3.1 Équivalent de `INKEY$` (routine `C90A`) . . . 6

3.2 Tracé de droites (commande `14` du T6834) . . . 6

3.3 Caractères graphiques (commande `1A`) . . . 6

3.4 Touches curseur (commande `02`, STICK) . . . 7

3.5 Test d'une touche (commande `28`, TEST CHR) . . . 7

3.6 Temporisation . . . 7

3.7 Musique (routine `C2DF`) . . . 7

3.8 `PSET`, `PRESET`, `POINT` (commandes `11`, `12`, `13`) . . . 8

3.9 Équivalent de `INPUT "message";A$` . . . 8

3.10 Équivalent de `INPUT "message";A%` . . . 9

3.11 Sortie sur la table traçante X-710 (routine `CEF7`) . . . 9

4 Extension des fonctions du BASIC (p. 41-45) **9**

4.1 Le crochet d'erreur . . . 9

4.2 Messages d'erreur en français . . . 10
4.3 Nouvelle instruction `CARTE` . . . 10
4.4 Détournement de `PAINT` . . . 11

2e partie : Hard **11**

5 Interruptions et entrées/sorties (p. 47-68) **11**
5.1 Les interruptions du X-07 (p. 47-51) . . . 11
5.2 Les entrées/sorties (p. 57-68) . . . 14

6 L'interface vidéo X-720 (p. 69-82) **18**
6.1 Matériel . . . 18
6.2 Logiciel . . . 18
6.3 Quatre utilitaires (p. 80-82) . . . 19

3e partie : Logiciel **20**

7 Présentation et chargement (p. 85) **20**
7.1 Récapitulatif des logiciels . . . 20

8 LMDATA et applications (p. 86-88) **20**

9 LOGOGÉNÈSE (p. 89-90) **23**

10 LLIST (p. 91-95) **24**

11 LABYRINTHE 3D (p. 96-100) **26**

12 Le SOLITAIRE (p. 101-106) **26**

13 Les PENTOMINOS (p. 107-113) **27**

14 AUTONUM (p. 114-116) **27**

15 REFBAS (p. 117-120) **28**

16 EXABAS (p. 121-124) **29**

17 LE PIÈGE (p. 125-137) **29**

18 OTHELLO-REVERSI (p. 138-144) **30**

19 Conclusion du livre (p. 145) **30**

Annexes **31**

20 Assembleurs et désassembleurs (annexe 1) **31**

21 Le Club C7 (annexe 2) **31**

22 Les mystères du X-07 (annexe 3) **31**

23 La cassette (annexe 4) **31**

24 Bibliographie (annexe 5) **31**

Présentation du livre (p. 1-10)

Auteur	Présentation (p. 2)
André Tonic	19 ans, études d'économie ; président du Club C7 et directeur de la publication des Éditions Neptune
Eddy Arevian	23 ans, électronicien ; conseiller technique du Club C7
Pierre Millet	23 ans ; trésorier du Club C7
Jean-Jacques Rousseau	43 ans ; maître de conférences à l'université du Maine

La préface est un court récit : un technicien de Kourou dépanne le lancement d'une fusée grâce à son X-07. L'introduction présente le livre comme la suite des *Mystères du X-07* : les notions de base de l'assembleur Z80 et les adresses de la ROM y sont supposées connues.

1re partie : Soft (p. 13-45)

1 Utilisation de routines en langage machine (p. 15-25)

Une routine en langage machine se lance depuis le BASIC de deux façons :

— `EXEC adresse`, sans argument ;
— `Z=USR(adresse,argument)`, avec un argument et une valeur de retour.

1.1 Où implanter la routine ? (p. 15-19)

Les pointeurs de début et de fin de la zone BASIC sont recalculés à chaque mise sous tension (sauf après un `SLEEP`). Il faut donc réserver la place de la routine. Les adresses dépendent de la quantité de mémoire (8, 12, 16, 20 ou 24 Ko).

Méthode	Principe	Avantages	Inconvénients
`CLEAR`	`CLEAR 50,&H4000` fixe la limite haute de la zone BASIC (MEMSIZ, `01DF`) ; les octets situés au-delà sont libres	classique, adresse fixe	à refaire à chaque mise sous tension
Fond des fichiers RAM	la routine est placée à la fin de la zone des fichiers RAM (RAMSTR, `0210` ; taille réglée par `FSET`)	pas de `CLEAR` à refaire	vérifier la place libre avec `DIR` ; ne rien écraser
Ligne `REM`	une ligne `REM` suivie d'autant de caractères que la routine, remplacés par `POKE`	tous les codes admis ; adresse absolue ; code résident	80 codes au plus (LCD) ; certains codes troublent l'interpréteur ; ligne impossible à éditer ou à lister
Chaîne dans le programme	`A$="TURLUTUTU"` : le code remplace les caractères	adresse absolue ; code résident	codes `00` et `22` (guillemet) interdits ; listage imprévisible
Zone des chaînes	`A$=STRING$(100,"*")`	tous les codes admis ; listage normal	relogeable uniquement ; adresse à recalculer avant chaque appel ; code à réimplanter à chaque exécution
Tableau d'entiers	le code est rangé dans un tableau `U%()`, deux octets par élément	passage facile de plusieurs arguments	octets croisés deux à deux ; relogeable uniquement ; mise en œuvre délicate

Calcul des adresses.

— Ligne `REM` n : après `RESTORE` n, le pointeur des DATA (`0328`–`0329`) désigne la ligne ; le premier caractère après `REM` se trouve 6 octets plus loin :
`AD=PEEK(&H329)*256+PEEK(&H328)+6`.
— Chaîne :
`AD=VARPTR(A$):AD=PEEK(AD+2)*256+PEEK(AD+1)`
(le descripteur contient la longueur, puis l'adresse, poids faible en premier).
— Tableau : `AD=VARPTR(U%(0))`.

Codage dans un tableau d'entiers. Un entier est rangé poids faible en premier : les octets `00,21` s'écrivent donc `2100` (8448). Il faut souvent ajouter des `NOP` de cadrage pour que les données (adresses, longueurs) tombent sur un élément entier. Exemple : une copie de bloc par `LDIR` devient `NOP / LD HL,source / NOP / LD DE,dest / NOP / LD BC,long / LDIR / RET / NOP` :

Élément	Octets	Valeur	Rôle
EX(0)	00 21	8448	NOP ; LD HL,...
EX(1)	ss ss	-	adresse source
EX(2)	00 11	4352	NOP ; LD DE,...
EX(3)	dd dd	-	adresse destination
EX(4)	00 01	256	NOP ; LD BC,...
EX(5)	ll ll	-	longueur
EX(6)	ED B0	-20243	LDIR
EX(7)	C9 00	201	RET ; NOP

N.d.R. Le livre donne −20234 pour EX(6) à la p. 19. La bonne valeur, −20243 (`B0ED`), est celle qu'il utilise dans les exemples 4 et 5.

1.2 Appel et passage d'arguments (p. 19-21)

Sans argument. `EXEC adresse`. La routine doit se terminer par `RET`, sinon le X-07 se bloque et il faut un RESET.

Un argument : `Z=USR(adresse,argument)`.

— Argument chaîne : DE pointe sur le descripteur (longueur, puis adresse sur deux octets).
— Argument numérique : la valeur est dans l'accumulateur flottant (`044E`–`0455`) ; A contient le type (2 entier, 4 simple précision, 8 double précision) et HL pointe sur `044E`. Pour un entier, il se trouve en `0450`–`0451`.
— Valeur de retour : `USR` renvoie le contenu de l'accumulateur.

Étiq.	Instruction	Commentaire
	`INC HL`	HL pointe sur l'entier (`0450`)
	`INC HL`	
	`PUSH HL`	sauvegarde pour le retour (inutile si rien à renvoyer)
	`LD E,(HL)`	
	`INC HL`	
	`LD D,(HL)`	
	`EX DE,HL`	HL = argument entier
	`...`	traitement ; résultat entier dans BC
	`POP HL`	
	`LD (HL),C`	renvoi de BC dans l'accumulateur
	`INC HL`	
	`LD (HL),B`	
	`RET`	

Plusieurs arguments. On les range dans un tableau d'entiers dont on passe l'adresse :

```
T=USR(adresse,VARPTR(ARG(0)))
```

La routine récupère cette adresse (comme ci-dessus) puis la place dans IX par `PUSH DE / POP IX` ; les arguments se lisent alors en `(IX+0)`, `(IX+1)`, etc. Avec une implantation absolue, on peut aussi déposer les arguments par `POKE` avant l'appel.

N.d.R. Le livre écrit que `ARG(0)=aa00` donne `(IX+0)=aa` ; en mémoire, l'octet de poids faible vient en premier : `(IX+0)=00` et `(IX+1)=aa`. Le paragraphe est numéroté 1.1.2.2 au lieu de 1.2.2.2.

1.3 Exemples (p. 22-25)

Exemples 1 à 3 : édition automatique de la ligne fautive

En cas d'erreur, la routine passe directement en mode édition sur la ligne fautive (`LIST@`). Le même code de 20 octets est implanté de trois façons : ligne `REM`, chaîne de la zone des chaînes, tableau d'entiers.

Adr.	Octets	Étiq.	Instruction	Commentaire
00	06 08		LD B,$08	
02	E1	L	POP HL	vide 8 niveaux de pile
03	10 FD		DJNZ L	
05	2A 15 03		LD HL,($0315)	numéro de la ligne en erreur
08	E5		PUSH HL	
09	D1		POP DE	
0A	CD 0B F3		CALL $F30B	recherche de la ligne
0D	FD 21 00 40		LD IY,$4000	
11	C3 8A FE		JP $FE8A	édition (LIST@)

```
10 GOTO 20
15 REM01234567890123456789
20 RESTORE 15:AD%=PEEK(&H329)*256+PEEK(&H328)+6
30 FOR I%=0 TO 19:READ B$:POKE AD%+I%,VAL("&H"+B$):NEXT I%
40 DATA 06,08,E1,10,FD,2A,15,03,E5,D1,CD,0B,F3,FD,21,00,40,C3,8A,FE
50 ON ERROR GOTO 65000
60 ' DEBUT DU PROGRAMME
100 A=5:B=0:C=A/B
64999 END
65000 PRINT "Erreur"
65010 EXEC AD%
```

Variante « zone des chaînes » (exemple 2) : la ligne 15 disparaît, la ligne 10 devient `A$=STRING$(20,"#")` et l'adresse est calculée par `AD%=VARPTR(A$):AD%=PEEK(AD%+1)+256*PEEK(AD%+2)`, en ligne 20 puis de nouveau en ligne 65005, juste avant le `EXEC`.

Variante « tableau d'entiers » (exemple 3) : mêmes octets, croisés deux à deux.

```
10 DIM U%(9)
20 U%(0)=&H0806:U%(1)=&H10E1:U%(2)=&H2AFD:U%(3)=&H0315:U%(4)=&HD1E5
30 U%(5)=&H0BCD:U%(6)=&HFDF3:U%(7)=&H0021:U%(8)=&HC340:U%(9)=&HFE8A
50 ON ERROR GOTO 65000
100 D$=5
64999 END
65000 PRINT"Erreur"
65010 EXEC VARPTR(U%(0))
```

Exemple 4 : remplir un tableau avec une même valeur

La routine `LDIR` du § 1.1 recopie `A!(0)` dans tout le tableau : la source et la destination se chevauchent d'un élément. La longueur vaut `N*2` pour des entiers, `N*4` en simple précision et `N*8` en double précision.

```
10 DEFINT A-Z:N=50:DIM U(7),A!(N)
15 AD=0
20 U(0)=8448:U(2)=4352:U(4)=256:U(6)=-20243:U(7)=201
30 A!(0)=100:GOSUB 100
40 PRINT A!(0);A!(1);A!(20)
50 A!(0)=0:GOSUB 100
60 PRINT A!(0);A!(1);A!(20)
```

```
70 END
100 U(1)=VARPTR(A!(0)):U(3)=VARPTR(A!(1)):U(5)=N*4
110 EXEC VARPTR(U(0)):RETURN
```

Exemple 5 : remplir rapidement l'écran vidéo

Même principe, appliqué à la mémoire vidéo de la X-720 (`8000`). Il faut ajuster `US(5)` selon le mode d'écran ; le motif à reproduire est déposé en `8000` à la ligne 40.

```
5 SCREEN 2:CLS
6 LINE (0,0)-(200,180)
10 DEFINT A-Z:DIM US(7)
20 US(0)=8448:US(2)=4352:US(4)=256:US(6)=-20243:US(7)=201
30 US(1)=&H8000:US(3)=&H8001:US(5)=511
40 POKE &H8000,64
50 EXEC VARPTR(US(0))
70 END
```

Exemple 6 : `RESTORE` calculé

Le BASIC n'accepte pas de variable après `RESTORE`. La routine suivante (18 octets, relogeable) le permet : `Z=USR(AD,J)` équivaut à `RESTORE J`. Elle est utilisée par le logiciel LOGOGÉNÈSE (§ 9).

Adr.	Octets	Étiq.	Instruction	Commentaire
00	23		INC HL	
01	23		INC HL	
02	5E		LD E,(HL)	DE = numéro de ligne (argument)
03	23		INC HL	
04	56		LD D,(HL)	
05	CD 0D F3		CALL $F30D	recherche de la ligne : BC = adresse
08	60		LD H,B	
09	69		LD L,C	
0A	D2 38 F6		JP NC,$F638	ligne absente : « UL Error »
0D	2B		DEC HL	
0E	22 28 03		LD ($0328),HL	nouveau pointeur des DATA
11	C9		RET	

2 Codage des variables (p. 26-30)

2.1 Variables simples

`VARPTR(V)` pointe sur la valeur. Les trois octets qui précèdent contiennent le type et le nom.

Type (taille)	Exemple (en hexadécimal)	Codage
Entier % (5 octets)	`02 41 00 | D2 04`	`A%=1234` : type 2, nom « A », valeur poids faible en premier
Simple précision ! (7 octets)	`04 43 44 | 44 12 34 00`	`CD!=1234` : exposant + 40, puis 3 octets BCD (0,1234 E4)
Double précision # (11 octets)	`08 50 49 | 41 31 41 59 26 53 58 98`	`PI#=3.1415926535898` : exposant, puis 7 octets BCD
Chaîne $ (6 octets + texte)	`03 54 00 | 04 5B 05`	`T$="TOTO"` : longueur, puis adresse du texte

Les deux octets de nom contiennent les deux premières lettres ; le second vaut 0 pour un nom d'une seule lettre.

2.2 Tableaux

Devant `VARPTR(T(0))` se trouve un en-tête : type, nom (2 octets), taille jusqu'au tableau suivant (2 octets), nombre de dimensions, puis, pour chaque dimension en partant de la dernière, l'indice maximal + 1 (2 octets).

Les éléments sont rangés en faisant varier le premier indice le plus vite : `T(0,0,0)`, `T(1,0,0)`, `T(2,0,0)`, `T(0,1,0)`...

Déclaration	En-tête (hexadécimal)	Taille totale
`DIM T%(10)`	`02 54 00 19 00 01 0B 00`	
`DIM T!(10)`	`04 54 00 2F 00 01 0B 00`	
`DIM T%(4,5)`	`02 54 00 41 00 02 06 00 05 00`	70 octets
`DIM AT%(2,4,5)`	`02 41 54 BB 00 03 06 00 05 00 03 00`	192 octets
`DIM T%(20)`		50 octets
`DIM T!(20)`		92 octets
`DIM T#(20)`		176 octets
`DIM T#(4,5)`		250 octets
`DIM T#(2,4,5)`		732 octets

3 Routines de la ROM (p. 31-40)

Ces routines courtes utilisent les adresses des *Mystères du X-07*. Elles sont données sous forme de source, à implanter où l'on veut.

3.1 Équivalent de `INKEY$` (routine `C90A`)

Étiq.	**Instruction**	**Commentaire**
`INKEY`	`XOR A`	
	`CALL $C90A`	lecture d'une touche
	`JR Z,INKEY`	aucune touche : on recommence
	`RET`	A = code ASCII

3.2 Tracé de droites (commande `14` du T6834)

HL pointe sur une table de coordonnées (X_i, Y_i, X_f, Y_f) ; B est le nombre de droites. La routine `C92F` fait avancer HL toute seule.

Étiq.	**Instruction**	**Commentaire**
`DROITE`	`PUSH BC`	
	`LD A,$14`	commande LINE
	`LD BC,$0400`	4 paramètres, aucune réponse
	`CALL $C92F`	
	`POP BC`	
	`DJNZ DROITE`	
	`RET`	

3.3 Caractères graphiques (commande `1A`)

Codes `80`–`9F` et `E0`–`FF`. La table contient, pour chaque caractère, son code suivi de ses 8 octets de motif.

Étiq.	**Instruction**	**Commentaire**
`CRECAR`	`LD B,nbre`	nombre de caractères
	`LD HL,TCAR`	table : code, 8 octets, code, ...
`CC`	`PUSH BC`	
	`LD A,$1A`	commande UDC WRITE
	`LD BC,$0900`	9 paramètres, aucune réponse
	`CALL $C92F`	
	`POP BC`	
	`DJNZ CC`	
	`RET`	

3.4 Touches curseur (commande 02, STICK)

Étiq.	Instruction	Commentaire
FLECHE	CALL $C0BD	vide les tampons du clavier
	LD A,$82	commande 2 ; bit 7 à 1 : aucun paramètre
	LD BC,$0001	0 paramètre, 1 octet de réponse
	PUSH DE	
	LD DE,BUF	tampon de réponse
	CALL $C92F	
	LD A,(DE)	
	POP DE	
	CP $33	droite
	JR Z,DROITE	
	CP $37	gauche
	JR Z,GAUCHE	
	CP $35	bas
	JR Z,BAS	
	CP $31	haut
	JR Z,HAUT	
	RET	30 : aucune touche

N.d.R. Les *Mystères* (§ 7.6.1) donnent 32 pour droite et 36 pour bas ; ce livre utilise 33 et 35, et il a raison : le guide technique officiel de la carte XP-140 (tableau 6) donne 31 haut, 32 haut-droite, 33 droite, 34 bas-droite, 35 bas, 36 bas-gauche, 37 gauche, 38 haut-gauche.

3.5 Test d'une touche (commande 28, TEST CHR)

On charge A avec le code de la touche ; au retour, Z = 1 si elle est enfoncée.

Étiq.	Instruction	Commentaire
TOUCHE	LD HL,BUF+4	
	LD (HL),A	code de la touche
	LD A,$28	
	LD BC,$0101	1 paramètre, 1 octet de réponse
	LD DE,BUF+2	
	CALL $C92F	
	LD A,(DE)	0 si enfoncée, sinon FF
	OR A	positionne Z
	RET	

3.6 Temporisation

BC contient la durée.

Étiq.	Instruction	Commentaire
DELAY	DEC BC	
	LD A,B	
	OR C	
	JR NZ,DELAY	
	RET	

N.d.R. Le livre écrit DEC B : C n'est alors jamais décrémenté, et la boucle ne s'arrête jamais si C n'est pas nul. Il faut DEC BC.

3.7 Musique (routine C2DF)

IX pointe sur la mélodie : note, durée, note, durée… ; FF termine la mélodie et la note 0 est un silence (bruit). Tous les registres sont modifiés.

Étiq.	Instruction	Commentaire
MUSIC	LD A,(IX+0)	
	CP $FF	fin de la mélodie ?
	RET Z	
	LD E,A	note
	LD B,(IX+1)	durée
	LD D,0	
	LD C,D	
	INC IX	
	INC IX	
	LD HL,RTMU	adresse de retour pour C2DF
	PUSH HL	
	PUSH BC	
	PUSH DE	
	POP IY	IY = note (tonalité)
	POP HL	H = durée, L = 0
	CALL $C2DF	
RTMU	LD HL,($0E)	compteur de la note en cours
	LD A,H	
	OR L	
	JR NZ,RTMU	attente de la fin de la note
	JR MUSIC	

Mélodie d'exemple :

```
AIR1  DB 17,4,17,4,17,4,13,16,0,16,15,4
      DB 15,4,15,4,12,16,0,16,17,4,17,4
      DB 17,4,17,4,13,4,18,4,18,4,18,4
      DB 17,4,25,4,25,4,25,4,22,16,255
```

N.d.R. Les commentaires du livre sont inversés (« IY contient la durée », « HL la note »). Le code est pourtant conforme aux *Mystères* : C2DF attend la tonalité dans IY et la durée dans H.

3.8 PSET, PRESET, POINT (commandes 11, 12, 13)

Étiq.	Instruction	Commentaire
SET	LD A,$11	PSET
	JR SET1	
POINT	LD A,$13	
	JR SET1	
RESET	LD A,$12	PRESET
SET1	LD HL,BUF	BUF contient X et Y
	LD BC,$0200	2 paramètres, aucune réponse
	JP $C92F	

N.d.R. D'après les *Mystères*, la commande 13 est PEOR (inversion du point). Le vrai POINT est la commande 10, qui renvoie un octet : il faudrait alors `LD BC,$0201`.

3.9 Équivalent de INPUT "message";A$

La routine FEF7 affiche le message, EBF2 lit le clavier. La chaîne saisie (20 caractères au plus) est copiée dans BUF et sa longueur dans LONG ; une réponse vide fait recommencer.

Étiq.	Instruction	Commentaire
ACQMES	LD HL,MESSAGE	message terminé par 0
	CALL $FEF7	
	CALL $EBF2	saisie ; HL = tampon −1

Étiq.	Instruction	Commentaire
	INC HL	
	LD DE,BUF	
	LD C,0	
ACQ	LD A,(HL)	
	LD (DE),A	
	OR A	
	JR Z,ACQ2	fin de la saisie
	INC HL	
	INC DE	
	INC C	
	LD A,C	
	CP 20	
	JR C,ACQ	
ACQ2	LD A,C	
	CP 0	
	JR Z,ACQMES	chaîne vide : on recommence
	LD (LONG),A	
	RET	

3.10 Équivalent de `INPUT "message";A%`

Étiq.	Instruction	Commentaire
ACQNUM	LD HL,MESSAGE	
	CALL $FEF7	
	CALL $EBF2	saisie
	RST $10	premier caractère
	CALL $F595	conversion ASCII → entier dans DE

N.d.R. Vérifié dans l'émulateur (vraie ROM) : `F595` est la routine qui lit un **numéro de ligne**. Elle ne lit qu'un entier **non signé** de 0 à 65535 : une saisie de `-56` donne 0, et `40000` est accepté sans erreur `OV`. Ce n'est donc un équivalent de `INPUT A%` que pour les nombres positifs. Les espaces de tête sont bien ignorés (RST 10).

3.11 Sortie sur la table traçante X-710 (routine `CEF7`)

Étiq.	Instruction	Commentaire
	LD HL,CHAINE	
LPRINT	LD A,(HL)	
	OR A	fin de chaîne ?
	RET Z	
	PUSH HL	
	CALL $CEF7	envoi à la X-710
	POP HL	
	INC HL	
	JR LPRINT	

Chaîne d'exemple, en décimal : `18,83,48,13,67,49,77,48,45,54,13,65,13,10,00`. Le code 18 fait passer en mode graphique ; « S0 » règle la taille, « C1 » la couleur ; « M0,-6 » remonte de 6 points ; 65 est la lettre A ; 13 et 10 sont CR et LF ; 0 termine la chaîne.

4 Extension des fonctions du BASIC (p. 41-45)

4.1 Le crochet d'erreur

Avant d'afficher une erreur, la ROM exécute l'instruction placée en `00AB` (normalement `RET`, `C9`). On y met un `JP xxxx` vers sa propre routine, qui reçoit le numéro d'erreur dans E. Si elle ne traite pas l'erreur, elle rend la

main par RET et l'erreur suit son cours normal. Codes utiles : 2 = erreur de syntaxe, 6 = débordement, 8 = numéro de ligne indéfini.

4.2 Messages d'erreur en français

Implantation	1C00–1C8C
Longueur	141 octets
Lancement	EXEC &H1C00 (installe le détournement)

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	3E C3		LD A,$C3	code de JP
1C02	32 AB 00		LD ($00AB),A	
1C05	21 0C 1C		LD HL,DB	
1C08	22 AC 00		LD ($00AC),HL	JP DB en 00AB
1C0B	C9		RET	
1C0C	7B	DB	LD A,E	numéro d'erreur
1C0D	32 00 1B		LD ($1B00),A	(absent du source du livre)
1C10	FE 02		CP 2	syntaxe ?
1C12	28 09		JR Z,E2	
1C14	FE 06		CP 6	débordement ?
1C16	28 11		JR Z,E6	
1C18	FE 08		CP 8	ligne indéfinie ?
1C1A	28 19		JR Z,E8	
1C1C	C9		RET	autre erreur : traitement normal
1C1D	21 41 1C	E2	LD HL,SY	
1C20	CD F7 FE		CALL $FEF7	affiche le message
1C23	CD 79 1C		CALL NO	affiche « LIGNE n »
1C26	C3 3D F2		JP $F23D	arrêt, attente curseur
1C29	...	E6	...	idem avec DE (1C56)
1C35	...	E8	...	idem avec ID (1C67)
1C41	...	SY	DEFM	« ERREUR DE SYNTAXE », CR LF, 0
1C56	...	DE	DEFM	« DEBORDEMENT EN », CR LF, 0
1C67	...	ID	DEFM	« LIGNE INDEFINIE », CR LF, 0
1C79	21 86 1C	NO	LD HL,N1	
1C7C	CD F7 FE		CALL $FEF7	« LIGNE »
1C7F	2A DB 01		LD HL,($01DB)	dernière ligne exécutée
1C82	CD 98 BB		CALL $BB98	affichage décimal
1C85	C9		RET	
1C86	...	N1	DEFM	« LIGNE », 0

```
10 REM Messages d'erreurs en francais
20 FOR I=&H1C00 TO &H1C00+140
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT
50 PRINT "EXEC &H1C00 sera necessaire au detournement des futures erreurs"
60 DATA 3E,C3,32,AB,0,21,C,1C,22,AC,0,C9,7B,32,0,1B,FE,2,28,9,FE,6,28,11,FE,8
70 DATA 28,19,C9,21,41,1C,CD,F7,FE,CD,79,1C,C3,3D,F2,21,56,1C,CD,F7,FE,CD,79,1C
80 DATA C3,3D,F2,21,67,1C,CD,F7,FE,CD,79,1C,C3,3D,F2,45,52,52,45,55,52,20,44
90 DATA 45,20,53,59,4E,54,41,58,45,20,A,D,0,44,45,42,4F,52,44,45,4D,45,4E,54,20
100 DATA 45,4E,A,D,0,4C,49,47,4E,45,20,49,4E,44,45,46,49,4E,49,45,A,D,0,21,86
110 DATA 1C,CD,F7,FE,2A,DB,1,CD,98,BB,C9,4C,49,47,4E,45,20,0
```

N.d.R. Les DATA du livre contiennent trois fautes dans les messages : « ERREURE DE SYMTAXE », et un code 04 au lieu de 44 (« D ») dans « DEBORDEMENT ». Elles sont corrigées ci-dessus sans changer la longueur des messages (« ERREUR DE SYNTAXE » suivi d'une espace), donc sans décaler les adresses.

4.3 Nouvelle instruction CARTE

Taper CARTE produit une erreur de syntaxe (E = 2). La routine relit alors le mot à rebours (HL pointe juste après) et, s'il s'agit bien de CARTE, saute en C4D1 (« CREATE SYSTEM ? », effacement de la carte mémoire).

Implantation	1C00–1C33
Longueur	52 octets
Lancement	EXEC &H1C00, puis taper CARTE

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	3E C3 ...		...	installation du JP en 00AB (comme ci-dessus)
1C0C	7B	DB	LD A,E	
1C0D	FE 02		CP 2	erreur de syntaxe ?
1C0F	C0		RET NZ	
1C10	CD 31 1C		CALL PR	caractère précédent
1C13	FE 45		CP "E"	
1C15	C0		RET NZ	
	...		...	idem pour T, R, A, C
1C2E	C3 D1 C4		JP $C4D1	« CREATE SYSTEM ? »
1C31	2B	PR	DEC HL	
1C32	7E		LD A,(HL)	
1C33	C9		RET	

```
10 ' Instruction CARTE
20 FOR I=&H1C00 TO &H1C00+51
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT
50 EXEC &H1C00
60 DATA 3E,C3,32,AB,0,21,C,1C,22,AC,0,C9,7B,FE,2,C0,CD,31,1C,FE,45,C0,CD,31,
   1C,FE,54,C0,CD,31,1C,FE,52,C0,CD,31,1C,FE,41,C0,CD,31,1C,FE,43,C0,C3,D1,
   C4,2B,7E,C9
```

4.4 Détournement de PAINT

Sans la X-720, PAINT passe par le saut placé en 0099 (qui mène à SN ERROR). On peut le détourner de la même manière :

Étiq.	Instruction	Commentaire
	ORG $1C00	
	LD A,$C3	
	LD ($99),A	
	LD HL,DB	
	LD ($9A),HL	JP DB en 0099
	RET	
DB	PUSH HL	
	LD HL,ME	
	CALL $D5B0	affiche « PAINT »
	POP HL	
	RET	
ME	DEFM PAINT	
	DEFB $00	

2e partie : Hard (p. 46-82)

5 Interruptions et entrées/sorties (p. 47-68)

5.1 Les interruptions du X-07 (p. 47-51)

Une interruption permet à un périphérique de signaler un événement sans que le programme ait à le surveiller en permanence. Le Z80 la prend en compte à la fin de l'instruction en cours : il empile l'adresse de retour (PC), exécute la routine de traitement puis reprend. DI et EI interdisent et autorisent les interruptions masquables.

Ligne	Source	Vecteur	Routine ROM
INT A	sous-processeur T6834 (clavier…)	003C	JP C799
INT B	interface série	0034	JP C7A3
INT C	X-720 seulement : horloge du curseur	002C	RET (C9)
NMI	non utilisée sur le X-07 ; non masquable, priorité maximale	0066	RET (C9)

Ces vecteurs sont en RAM (zone système) : on peut les modifier. NMI, qui ne fait rien, est la plus facile à détourner.

N.d.R. Au § 5.1.4, le livre dit que la ligne B vient du sous-processeur, ce qui contredit le § 5.1.2 (INT A = T6834, INT B = série).

Déclencher une NMI à la main (p. 51-53)**.** Les bornes INT A et NMI sont accessibles sur le connecteur d'extension arrière (brochage p. 127 du guide utilisateur). NMI (broche 20) est active à l'état haut, INT A à l'état bas. Le +5 V est sur les broches 39 et 40. Un bref contact entre les broches 20 et 39 déclenche une NMI. Les auteurs conseillent un bouton-poussoir entre ces deux broches : un contact trop long fausse le fonctionnement.

Rangée haute (paires)	2	4	…	20 NMI	…	40 +5V
Rangée basse (impaires)	1	3	…	19	…	39 +5V

Pour aller plus loin. En divisant le signal CLOCK (broche 29) jusqu'à 5 à 10 Hz et en le renvoyant sur la broche 20, on obtient un « top » périodique qui permet un multitâche simple. Un montage anti-rebond (bascule) est recommandé. Applications envisagées : détection de niveaux, surveillance, sonnerie de téléphone, disquettes, DMA.

Expérience 1 : un BEEP à chaque NMI

Le BASIC compte de 1 à 10 000 pendant que chaque NMI émet un bip (caractère BELL, code 7).

Implantation	1B00–1B12
Longueur	19 octets
Lancement	EXEC &H1B00, puis relier brièvement les broches 20 et 39

Adr.	Octets	Étiq.	Instruction	Commentaire
1B00	3E C3		LD A,$C3	
1B02	32 66 00		LD ($0066),A	
1B05	21 0C 1B		LD HL,DB	
1B08	22 67 00		LD ($0067),HL	JP DB en 0066
1B0B	C9		RET	
1B0C	3E 07	DB	LD A,$07	BELL
1B0E	CD BE C1		CALL $C1BE	sortie console
1B11	ED 4D		RETI	

```
5 REM LE BEEP
10 FOR I=&H1B00 TO &H1B12
20 READ A$:POKE I,VAL("&H"+A$)
30 NEXT
35 EXEC &H1B00
40 CLS:PRINT"COURT-CIRCUITEZ LES BROCHES"
50 FOR I=1TO10000
60 LOCATE10,2:PRINTI
70 NEXT
100 DATA 3E,C3,32,66,00,21,C,1B
101 DATA 22,67,0,C9
102 DATA 3E,7,CD,BE,C1,ED,4D
```

N.d.R. Le listing du livre ne contient pas de `EXEC &H1B00` : la ligne 35 est ajoutée ici. Une NMI doit normalement se terminer par `RETN` (`ED 45`), qui restaure l'état des interruptions ; le livre emploie `RETI` (`ED 4D`).

N.d.R. Vérifié dans l'émulateur (vraie ROM) : la routine écrase le registre A (`LD A,$07`) sans le sauvegarder. Le BASIC interrompu est faussé : avec des NMI répétées, le compteur repart en arrière et un `Break in 60` intempestif arrête le programme. Il faut encadrer la routine par `PUSH AF` et `POP AF` : `1B0C F5 / 3E 07 / CD BE C1 / F1 / ED 45`, soit `102 DATA F5,3E,7,CD,BE,C1,F1,ED,45` et `FOR I=&H1B00 TO &H1B14` en ligne 10. Ainsi corrigée, chaque NMI donne un bip et le comptage reste juste. L'expérience 2, qui sauvegarde tous ses registres, fonctionne telle quelle.

Expérience 2 : deux routines en langage machine

Une boucle affiche les codes 0 à 255 ; chaque NMI affiche « INTERUPTION !!!! » sans arrêter la boucle.

Implantation	`1A00`–`1A0B` et `1B00`–`1B4C`
Longueur	12 + 77 octets
Lancement	`EXEC &H1A00:EXEC &H1B00`

Adr.	Octets	Étiq.	Instruction	Commentaire
1A00	3E C3		LD A,$C3	installation :
1A02	32 66 00		LD ($0066),A	JP 1B20 en 0066
1A05	21 20 1B		LD HL,$1B20	
1A08	22 67 00		LD ($0067),HL	
1A0B	C9		RET	
1B00	3E 00		LD A,0	1er programme
1B02	3C	B1	INC A	
1B03	21 02 05		LD HL,$0502	position du curseur
1B06	22 B8 00		LD ($00B8),HL	
1B09	CD BE C1		CALL $C1BE	affiche le caractère A
1B0C	CD 11 1B		CALL TP	
1B0F	18 F1		JR B1	
1B11	F5	TP	PUSH AF	temporisation
1B12	C5		PUSH BC	
1B13	3E 7F		LD A,$7F	
1B15	06 FF	B2	LD B,$FF	
1B17	05	B3	DEC B	
1B18	20 FD		JR NZ,B3	
1B1A	3D		DEC A	
1B1B	20 F8		JR NZ,B2	
1B1D	C1		POP BC	
1B1E	F1		POP AF	
1B1F	C9		RET	
1B20	F5 C5 D5 E5	DB	PUSH AF...HL	2e programme (NMI)
1B24	21 01 01		LD HL,$0101	
1B27	22 B8 00		LD ($00B8),HL	
1B2A	21 3C 1B		LD HL,ME	
1B2D	CD F7 FE		CALL $FEF7	affiche le message
1B30	CD 11 1B		CALL TP	
1B33	CD 9E CE		CALL $CE9E	efface l'écran
1B36	E1 D1 C1 F1		POP HL...AF	
1B3A	ED 4D		RETI	
1B3C	...	ME	DEFM	« INTERUPTION !!!! », 0

N.d.R. Vérifié dans la ROM : `00B8` contient la ligne et `00B9` la colonne, comptées à partir de 1. `LD HL,$0502` : `LD ($00B8),HL` place donc le curseur en colonne 5, ligne 2, et `$0101` le ramène en haut à gauche.

```
10 REM LES 2
20 FOR I=&H1A00 TO &H1A0B
30 READ A$:POKE I,VAL("&H"+A$):NEXT
50 RESTORE 700
60 FOR I=&H1B00 TO &H1B4C
70 READ A$:POKE I,VAL("&H"+A$):NEXT
90 EXEC&H1A00:EXEC&H1B00
```

```
500 DATA 3E,C3,32,66,00,21,20,1B,22
501 DATA 67,0,C9
700 DATA 3E,0,3C,21,2,5,22,B8,0
701 DATA CD,BE,C1,CD,11,1B
710 DATA 18,F1,F5,C5,3E,7F,6,FF,5
711 DATA 20,FD,3D,20,F8
720 DATA C1,F1,C9,F5,C5,D5,E5,21,1
721 DATA 1,22,B8,0,21
730 DATA 3C,1B,CD,F7,FE,CD,11,1B,CD
731 DATA 9E,CE,E1,D1
740 DATA C1,F1,ED,4D,49,4E,54,45,52
741 DATA 55,50,54,49,4F,4E,20,21
750 DATA 21,21,21,0
```

5.2 Les entrées/sorties (p. 57-68)

Le X-07 a quatre prises : magnétophone, série, parallèle et extension. Plutôt que de programmer directement les circuits, le livre réutilise les routines de la ROM qui servent à `INIT #n` (ouverture d'un fichier d'entrée/sortie).

Les dispositifs.

Nom	Table	Rôle et initialisation
KBD	`E778`	clavier et écran LCD ; l'ouverture vide le tampon du clavier
CASI	`E787`	entrée cassette
CASO	`E796`	sortie cassette
COM	`E7A4`	prise série ; vitesse et format à fournir
GPR	`E7B2`	table traçante X-710 (taille 2 ; CR et LF envoyés)
PRT	`E7C0`	imprimante série
RAM	`E7CE`	fichier en RAM (créé s'il n'existe pas, avec la taille demandée)
OPT	`E7DC`	coupleur optique
CON	`E7EA`	console (l'ouverture ne fait rien)
LPT	`E7F8`	imprimante parallèle Centronics (CR et LF)

Chaque entrée de la table de la ROM contient : le nom (par exemple « CASI : »), le numéro du dispositif, puis les adresses des sous-programmes de sortie d'un octet, d'entrée d'un octet, d'initialisation, et éventuellement un sous-programme optionnel. L'adresse indiquée ci-dessus pointe sur le numéro du dispositif. Exemple pour CASI : `43 41 53 49 BA | 00 02 | 12 E5 | 0A E0 | 35 DE | 0F E1`.

Routines utilisées.

Adresse	Rôle
`E6A8`	ouverture : DE = table du dispositif, IY = entrée de la table des fichiers ouverts (`02C5` pour le 1er, `02CD` pour le 2e), A = 0. Pour COM, B = mode, IX = vitesse
`E827`	assignation : DE = `02C5` (ou `02CD`) choisit le fichier courant
`E88F`	sortie de A vers le fichier courant (= RST `28`)
`E8D4`	entrée d'un octet du fichier courant dans A (ne rend la main que si une donnée est présente ; contient l'appel ABORT)

La table des fichiers ouverts (`02C5`–`02E8`) contient 5 entrées de 8 octets : adresse du dispositif et sauvegarde de HL, DE et BC. On peut donc ouvrir 5 fichiers au plus.

Sortie sur cassette (CASO)

Le moteur du magnétophone n'est pas commandé automatiquement : il faut mettre à 1 le bit 0 (REM) du port `F4`. Les données sont précédées de trois `00` et suivies de trois `FF`.

Implantation	`1C00`–`1C67`
Longueur	104 octets (données de `1C61` à `1C67`)
Lancement	`EXEC &H1C00`

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	11 96 E7		LD DE,$E796	table CASO
1C03	FD 21 C5 02		LD IY,$02C5	fichier #1
1C07	3E 00		LD A,0	
1C09	CD A8 E6		CALL $E6A8	INIT #1,"CASO :"
1C0C	DB F4		IN A,($F4)	
1C0E	CB C7		SET 0,A	moteur en marche
1C10	D3 F4		OUT ($F4),A	
1C12	CD 54 1C		CALL TP	temporisation
1C15	CD 54 1C		CALL TP	
1C18	11 C5 02		LD DE,$02C5	
1C1B	CD 27 E8		CALL $E827	assignation
1C1E	...		3 × (LD A,0 ; CALL $E88F)	trois 00 de début
1C2D	21 61 1C		LD HL,DD	données
1C30	7E	ST	LD A,(HL)	
1C31	FE 00		CP 0	fin des données ?
1C33	28 06		JR Z,TE	
1C35	CD 8F E8		CALL $E88F	envoi de l'octet
1C38	23		INC HL	
1C39	18 F5		JR ST	
1C3B	...	TE	3 × (LD A,$FF ; CALL $E88F)	trois FF de fin
1C4A	CD 54 1C		CALL TP	
1C4D	DB F4		IN A,($F4)	
1C4F	CB 87		RES 0,A	moteur arrêté
1C51	D3 F4		OUT ($F4),A	
1C53	C9		RET	
1C54	C5	TP	PUSH BC	temporisation
1C55	06 7F		LD B,$7F	
1C57	0E FF	E1	LD C,$FF	
1C59	0D	E2	DEC C	
1C5A	20 FD		JR NZ,E2	
1C5C	05		DEC B	
1C5D	20 F8		JR NZ,E1	
1C5F	C1		POP BC	
1C60	C9		RET	
1C61	01 ...06 00	DD	DEFB	données 1 à 6, puis 0

```
10 FOR I=&H1C00 TO &H1C00+103
20 READ A$:POKE I,VAL("&H"+A$)
30 NEXT I
40 PRINT "EXEC &H1C00 POUR DEMARRER LE PROGRAMME"
50 DATA 11,96,E7,FD,21,C5,2,3E,0,CD,A8,E6,DB,F4,CB,C7,D3,F4,CD,54,1C,CD,54,1C,
   11,C5,2,CD,27,E8,3E,0,CD,8F,E8,3E,0,CD,8F,E8,3E,0,CD,8F,E8,21,61,1C,7E,
   FE,0,28,6,CD,8F,E8,23,18,F5,3E,FF,CD,8F,E8,3E,FF,CD,8F,E8,3E,FF,CD,8F,E8,
   CD,54,1C,DB,F4,CB,87,D3,F4,C9,C5,6,7F,E,FF,D,20,FD,5,20,F8,C1,C9,1,2,3,4,
   5,6,0
```

N.d.R. Le premier listing de la p. 60 donne `LD DE,$E987` pour la table CASO : c'est `E796`. Le source porte `LD B,$00` dans la temporisation, les DATA `LD B,$7F`. Comme `00` sert de marqueur de fin, les données ne peuvent pas contenir d'octet nul.

Lecture sur cassette (CASI)

La routine attend les trois `00` de début, puis range les octets à partir de `1B00` jusqu'aux trois `FF` de fin. Aucune erreur de lecture n'est traitée.

Étiq.	Instruction	Commentaire
	LD DE,$E787	INIT #1,"CASI :"
	LD IY,$02C5	
	LD A,0	
	CALL $E6A8	

Étiq.	Instruction	Commentaire
	IN A,($F4)	moteur en marche
	SET 0,A	
	OUT ($F4),A	
	LD DE,$02C5	assignation
	CALL $E827	
E1	CALL $E8D4	attente des trois 00
	JR NZ,E1	(répété 3 fois)
	LD HL,$1B00	zone de réception
E2	CALL $E8D4	
	CP $FF	
	JR Z,FI	
	LD (HL),A	
	INC HL	
	JR E2	
FI	CALL $E8D4	2e FF ?
	...	(idem pour le 3e)
F2	IN A,($F4)	moteur arrêté
	RES 0,A	
	OUT ($F4),A	
	RET	

```
10 FOR I=&H1C00 TO &H1C00+81
20 READ A$:POKE I,VAL("&H"+A$)
30 NEXT
40 PRINT "EXEC &H1C00 pour demarrer ..."
50 DATA 11,87,E7,FD,21,C5,2,3E,0,CD,A8,E6,DB,F4,CB,C7,D3,F4,11,C5,2,CD,27,E8,
   CD,D4,E8,20,FB,CD,D4,E8,20,F6,CD,D4,E8,20,F1,21,0,1B,CD,D4,E8,FE,FF,28,4,
   77,23,18,F5,CD,D4,E8,FE,FF,28,4,7E,23,18,EA,CD,D4,E8,FE,FF,28,4,7E,23,18,
   DF,DB,F4,CB,87,D3,F4,C9
```

N.d.R. Les DATA du livre contiennent un octet parasite `8F` après le troisième `CALL $E8D4` : le dernier `JR` saute alors au milieu d'une instruction et la machine plante. Il est retiré ci-dessus (82 octets, `FOR ...+81`). La routine reste fragile : un `FF` isolé dans les données est remplacé par l'octet en mémoire (`LD A,(HL)` au lieu d'un rangement).

Clavier vers imprimante (KBD et GPR/LPT)

Chaque touche frappée est envoyée à l'imprimante ; CTRL N arrête.

Implantation	1C00–1C30
Longueur	49 octets
Lancement	EXEC &H1C00

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	11 F8 E7		LD DE,$E7F8	imprimante (voir N.d.R.)
1C03	FD 21 C5 02		LD IY,$02C5	fichier #1
1C07	3E 00		LD A,0	
1C09	CD A8 E6		CALL $E6A8	
1C0C	11 78 E7		LD DE,$E778	KBD
1C0F	FD 21 CD 02		LD IY,$02CD	fichier #2
1C13	3E 00		LD A,0	
1C15	CD A8 E6		CALL $E6A8	
1C18	11 CD 02	BL	LD DE,$02CD	clavier
1C1B	CD 27 E8		CALL $E827	
1C1E	CD D4 E8		CALL $E8D4	attente d'une touche
1C21	F5		PUSH AF	
1C22	11 C5 02		LD DE,$02C5	imprimante
1C25	CD 27 E8		CALL $E827	
1C28	F1		POP AF	
1C29	FE 0E		CP $0E	CTRL N : fin

Adr.	Octets	Étiq.	Instruction	Commentaire
1C2B	C8		RET Z	
1C2C	CD 8F E8		CALL $E88F	impression
1C2F	18 E7		JR BL	

```
10 FOR I=&H1C00 TO &H1C00+48
20 READ A$:POKE I,VAL("&H"+A$):NEXT
40 PRINT "EXEC &H1C00 pour demarrer ..."
50 DATA 11,F8,E7,FD,21,C5,2,3E,0,CD,A8,E6,11,78,E7,FD,21,CD,2,3E,0,CD,A8,E6,
   11,CD,2,CD,27,E8,CD,D4,E8,F5,11,C5,2,CD,27,E8,F1,FE,E,C8,CD,8F,E8,18,E7
```

N.d.R. Le source vise la X-710 (GPR, `E7B2`) et appelle par erreur `$F6A8` ; les DATA utilisent `E7F8` (LPT, imprimante parallèle) et `E6A8`. Pour la X-710, remplacer `F8,E7` par `B2,E7`.

Prise série : émission vers un Minitel

Pour COM, l'ouverture prend aussi le mode de l'ACIA dans B (ici « G ») et la vitesse dans IX (1200 bauds = `04B0`). Le code 12 (`0C`) efface l'écran du Minitel.

Implantation	`1C00`–`1C3F`
Longueur	64 octets
Lancement	`EXEC &H1C00`

Adr.	Octets	Étiq.	Instruction	Commentaire
1C00	11 A4 E7		LD DE,$E7A4	table COM
1C03	3E 00		LD A,0	
1C05	06 47		LD B,"G"	mode G de l'ACIA
1C07	DD 21 B0 04		LD IX,1200	vitesse
1C0B	FD 21 C5 02		LD IY,$02C5	
1C0F	CD A8 E6		CALL $E6A8	
1C12	11 C5 02		LD DE,$02C5	
1C15	CD 27 E8		CALL $E827	assignation
1C18	21 27 1C		LD HL,ME	
1C1B	7E	ET	LD A,(HL)	
1C1C	FE 00		CP 0	
1C1E	28 06		JR Z,FI	
1C20	CD 8F E8		CALL $E88F	émission
1C23	23		INC HL	
1C24	18 F5		JR ET	
1C26	C9	FI	RET	
1C27	0C ...	ME	DEFB 12 ; DEFM	effacement, « CA MARCHE BIEN COMME CA », 0

```
20 FOR I=&H1C00 TO &H1C00+63
30 READ A$:POKE I,VAL("&H"+A$):NEXT
50 PRINT " EXEC &H1C00 pour demarrer ..."
60 DATA 11,A4,E7,3E,0,06,47,DD,21,B0,4,FD,21,C5,2,CD,A8,E6,11,C5,2,CD,27,E8,
   21,27,1C,7E,FE,0,28,6,CD,8F,E8,23,18,F5,C9,C,43,41,20,4D,41,52,43,48,45,
   20,42,49,45,4E,20,43,4F,4D,4D,45,20,43,41,0
```

Prise série : réception

Même ouverture, puis lecture et affichage d'un caractère.

```
20 FOR I=&H1C00 TO &H1C00+30
30 READ A$:POKE I,VAL("&H"+A$):NEXT
50 PRINT " EXEC &H1C00 pour demarrer ..."
60 DATA 11,A4,E7,3E,0,6,47,DD,21,B0,4,FD,21,C5,2,CD,A8,E6,11,C5,2,CD,27,E8,
   CD,D4,E8,CD,BE,C1,C9
```

N.d.R. Le livre écrit `FOR ...+29` : le `RET` final (31e octet) n'est pas chargé. Il écrit aussi `CD,DE,C1` au lieu de `CD,BE,C1` (`CALL $C1BE`). Les deux erreurs sont corrigées ci-dessus. La routine ne lit qu'un seul caractère ; pour une réception continue, il faut boucler sur `CALL $E8D4 / CALL $C1BE`.

6 L'interface vidéo X-720 (p. 69-82)

6.1 Matériel

La X-720 relie le X-07 à un téléviseur par la prise Péritel ou par un modulateur. Les auteurs la jugent encombrante, non autonome, lente et peu servie en logiciels.

— Générateur vidéo **Motorola MC6847**.
— 6 Ko de RAM vidéo (`8000`–`97FF`), soit trois circuits 6117 de 2 Ko (IC203, IC204, IC205). Deux mémorisent l'état des points, le troisième les attributs (couleur).
— ROM : générateur de caractères et 4 Ko de complément du BASIC (`A000`–`AFFF`).
— Oscillateur couleur à 3,579545 MHz ; alimentation par adaptateur AD-5 et batterie NiCd.
— Connecteur vers le X-07 : NMI broche 20, CLK broche 29, VCC2 broches 39–40 (confirmé par le schéma de la fig. 11).

Ports d'entrée/sortie (fig. 9).

Ports	Sens	D7	D6	D5	D4	D3	D2	D1	D0
`80-8F`	lecture	données de la ROM de caractères							
`80-8F`	écriture	adresse dans la ROM de caractères							
`90-97`	lecture	FS	INT	–	–	B4	B3	B2	B1
`90-97`	écriture	FSR	A/G	+1K	GM2	GM1	GM0	CSS1	CSS0
`98-9F`	écriture	adresse basse d'une cartouche d'E/S							

En lecture : FS à l'état bas permet le défilement ; INT signale l'interruption du curseur ; B1–B4 ne servent pas. En écriture : FSR autorise l'interruption du curseur (0 : pas de curseur) ; A/G choisit le mode graphique ; +1K change la page vidéo affichée ; GM0–GM2 choisissent le mode graphique ; CSS0–CSS1 sélectionnent le jeu de couleurs.

Occupation de la RAM vidéo (fig. 10). En modes 1 et 2, seuls 512 octets d'écran (`8000`–`81FF`) et 512 octets d'attributs (`9000`–`91FF`) servent. En modes 3 et 4, l'écran occupe `8000`–`8BFF`. Les modes 5 et 6 utilisent les 6 Ko (`8000`–`97FF`). Le signal +1K commute la page affichée sans changer les adresses vues par le processeur, d'où des calculs d'adresse compliqués dans l'interpréteur et la lenteur de la X-720. Les auteurs conseillent de se placer dans un mode et d'écrire ses propres routines de tracé (voir LE PIÈGE en version vidéo).

Codage des points.

Mode	Contenu	Codage
1	texte seul, 16 lignes × 32 colonnes	1 octet par caractère (code ASCII), 512 octets
2	texte ou graphique	6 points par case et 2 bits de couleur
3 et 5	graphique	4 points par octet, 4 couleurs
4 et 6	graphique	8 points par octet, 1 bit par point

Le mode 4 utilise 3 Ko ; le mode 6 utilise 6 Ko, avec une définition doublée.

6.2 Logiciel

La ROM de la X-720 commence par le mot-clé `love`, détecté à la mise sous tension (voir les *Mystères*, § 7.4). Le sous-programme d'initialisation (`A0CA`) redirige les crochets de la zone système vers SCREEN, PAINT et COLOR, et étend PSET, PRESET, POINT, LINE et CIRCLE.

Instruction	Entrée	Instruction	Entrée	Instruction	Entrée
SCREEN	`AAD2`	SCREEN(X,Y)	`ABBD`	COLOR	`AA73`
PSET	`ABFB`	PRESET	`ABF6`	POINT	`AC14`
LINE	`AC86`	PAINT	`AD9B`	CIRCLE	`AED2`

Adresse	Variable système	Routine	Rôle
00B6	page active (0 = page 1)	A015	LSETCUR : curseur (comme C0A1)
00B7	page affichée	A050	INTC : clignotement du curseur
00D1	mode d'écran	A07C	ECRIRAM : écriture VRAM en cours ?
0015	0 caractères normaux, 1 caractères LCD	A08B	CONPUT : sortie écran
04D7	caractères KANA	A0CA	initialisation de la X-720
04E5	couleur d'avant-plan	A2C2	MODECR : changement de mode (A = 0 mode 1, 1 mode 2)
04E6	couleur de fond	A355	RETLCD : retour à l'écran LCD
04E7	palette (0 ou 2)	A374	VIDEO : affichage sur le téléviseur
04C6	X_i, Y_i de LINE	A4D1	PUTCAR : envoi d'un caractère
04C8	X_f, Y_f de LINE	A9FB-AA73	table des sous-programmes par mode
04CA	déplacement STEP	A631-A9FA	sous-programmes par mode

6.3 Quatre utilitaires (p. 80-82)

Test de présence. Écrit 80 en 8000 et relit ; sans X-720, saut en SN ERROR (F1AA).

```
10 REM TEST DE LA X-720
20 FOR I=&H1C00 TO &H1C0D
30 READ A$:POKE I,VAL("&H"+A$)
40 NEXT I
50 DATA 3E,80,32,00,80,3A,00,80,FE,80
60 DATA C2,AA,F1,C9
```

(LD A,$80 / LD ($8000),A / LD A,($8000) / CP $80 / JP NZ,$F1AA / RET)

Effacement partiel. Fige les 8 premières lignes de l'écran vidéo en réduisant la zone de défilement (00BB–00BD) et efface le reste (CE9E).

```
10 REM EFFACEMENT PARTIEL
20 FOR I=&H1C00 TO &H1C1D
30 READ A$:POKE I,VAL("&H"+A$):NEXT I
50 DATA CD,0D,1C,CD,9E,CE,21,08,01,22,B8,00,C9
60 DATA 3E,08,32,BB,00,3E,10,32,BC,00,3E,09,00
70 DATA 32,BD,00,C9
```

N.d.R. Le livre écrit LD A,(&16-8+1) et ses DATA donnent 3A,10,00 (LD A,($0010), lecture en mémoire) : il faut LD A,9 (3E,09). La ligne 60 est corrigée ici en 3E,09,00, où le 00 est un NOP qui garde les adresses inchangées.

SCREEN. Le mode moins 1 va dans C et en 00D1 ; la page active moins 1 dans D, la page affichée moins 1 dans E ; puis saut en AB09.

```
10 REM *** SCREEN ***
20 FOR I=&H1C00 TO &H1C0C
30 READ A$:POKE I,VAL("&H"+A$):NEXT I
50 DATA 3E,00,32,D1,00,4F,3E,01,57
60 DATA 5F,C3,09,AB
```

(LD A,0 / LD ($D1),A / LD C,A / LD A,1 / LD D,A / LD E,A / JP $AB09 : SCREEN 1, pages 2 et 2.)

COLOR. Écrit directement les couleurs d'avant-plan, de fond et la palette.

```
10 REM *** COLOR ***
20 FOR I=&H1C00 TO &H1C0F
30 READ A$:POKE I,VAL("&H"+A$):NEXT I
50 DATA 3E,01,32,E5,04,3E,02,32,E6,04
60 DATA 3E,00,32,E7,04,C9
```

3e partie : Logiciel (p. 84-145)

7 Présentation et chargement (p. 85)

Chaque logiciel est présenté avec ses spécifications, son principe, son mode d'emploi et son listing. Les logiciels en BASIC se chargent par `CLOAD` depuis la cassette des Éditions Neptune. Les logiciels en assembleur se chargent par `CLOAD` puis `RUN` (un chargeur BASIC lit les codes) ; `RUN 100` fait une copie des codes sur cassette.

Sans la cassette, chaque logiciel en assembleur est accompagné de deux programmes types, à adapter avec ses adresses de début `D` et de fin `F` :

```
5 REM *** ENTREUR DE CODES ***
10 CLEAR50,&H7FF:A=&H800
20 PRINT HEX$(A);" : ";:INPUT C$
30 V=VAL("&H"+C$):POKE A,V
49 A=A+1:IF A>&HA29 THEN PRINT"...":
   BEEP 2,3:END
50 GOTO20
```

```
10 CLEAR50,&H7FF
20 INIT#1,"CASI:"
30 INPUT#1,N$,D,F:MOTOR
50 PRINT"Trouve :";N$
60 FORI=D-1TOF
70 POKEI,INP(#1):NEXT:MOTOR:END
100 CLEAR50,&H7FF
110 D=&H800:F=&HA29
120 N$="LLIST":INIT#1,"CASO:"
130 INPUT"Magneto OK";T$
140 PRINT#1,N$,D,F:MOTOR
150 FORI=1TO1800:NEXT
160 FORI=DTOF:OUT#1,PEEK(I)
170 NEXT:MOTOR:END
```

L'entreur de codes (à gauche) fait taper les octets un par un, en hexadécimal, à partir des vidages du livre. Le chargeur (à droite) relit les codes sur cassette (lignes 10–90) ou les y enregistre (lignes 100–180).

La lecture commence en `D-1`, un octet plus tôt que l'écriture. Ce n'est probablement pas une erreur : le premier octet lu est sans doute le séparateur laissé par `PRINT#1` après `F`, qui tombe ainsi juste avant le programme. Les données arrivent donc bien de `D` à `F`.

7.1 Récapitulatif des logiciels

Logiciel	Type	Longueur	Implantation	Lancement
LMDATA	BASIC	1670 octets	-	RUN
LOGO CANON	BASIC	831 octets	-	RUN
RESTORE calculé	BASIC + LM	372 octets	2000	RUN
COPIE RAPIDE	BASIC + LM	539 octets	chaîne	RUN
LOGOGÉNÈSE	BASIC + LM	1819 octets	chaîne	RUN
LLIST	LM	554 octets	0800-0A29 (relogé)	EXEC &H800
LABYRINTHE 3D	LM	2066 octets	0800-1012	EXEC &H800
SOLITAIRE	LM	4609 octets	0800-11D5	EXEC &H800
PENTOMINOS	LM	3489 octets	0800-159A	EXEC &H800
AUTONUM	BASIC + LM	1093 + 216 octets	relogeable	RUN
REFBAS	LM	985 octets	3C10-3FE9	EXEC &H3C10
EXABAS	LM	901 octets	3C10-3F95	EXEC &H3C10
LE PIÈGE (LCD)	LM	4205 octets	0800-186C	EXEC &H800
LE PIÈGE (vidéo)	LM	4329 octets	0800-18E8	EXEC &H800
OTHELLO-REVERSI	LM	3073 octets	0800-1400	EXEC &HE00

8 LMDATA et applications (p. 86-88)

But. Transcrire automatiquement une routine en mémoire sous forme de lignes `DATA` décimales (trois chiffres par octet), à la suite du programme. On évite ainsi de recopier les codes à la main.

Utilisation. Implanter la routine, lancer LMDATA, donner les adresses de début et de fin. Effacer ensuite les lignes DATA superflues, puis les lignes 1 à 230 (commande DEL de la carte XP-140, ou à la main). Au-delà de 256 octets, il faut ajouter des lignes DATA. Faire une copie de sécurité avant.

Principe. Les lignes 10000 à 10150 contiennent des DATA « fictifs » (« ...») ; LMDATA y écrit directement, par `POKE`, les codes ASCII des chiffres (`CA=48` est le code de « 0 »). La ligne 40 calcule l'adresse du premier champ de la ligne 10000 ; la ligne 210 décompose chaque octet en centaines, dizaines et unités.

```
1 REM Ecriture automatique de codes LM en DATA
10 CLEAR50,&H1F00:DEFINTA-Z:LL=16:CA=48:CLS
20 INPUT"Adres. debut routine";DM:DD=DM
30 INPUT"Adresse fin routine";FM:NC=FM-DM+1:JC=INT(NC/LL)
40 RESTORE10000:AD=PEEK(&H328)+256*PEEK(&H329)+6
50 LB=LL:FORJ=1TOJC
55 PRINT"LIGNE";10000+10*J
60 GOSUB200:AD=AD+5:NEXTJ
70 LB=NC-JC*LL:IFLB=0THEN100
80 GOSUB200
90 PRINT"Effacer la fin de la ligne";10000+10*JC
100 PRINT"Effacer les lignes";10000+10*(JC+1);"";10150;
110 IFINKEY$=""THEN110
120 END
200 FORI=1TOLB:VC=PEEK(DM):DM=DM+1
210 C=INT(VC*.01):D=INT((VC-100*C)/10):U=VC-10*D-100*C
220 POKEAD,C+CA:POKEAD+1,D+CA:POKEAD+2,U+CA:AD=AD+4
230 NEXT:RETURN
10000 DATA...,...,...,...,...,...,...,...,...,...,...,...,...,...,...,...
  (idem jusqu'a la ligne 10150)
```

N.d.R. La ligne 55 annonce la ligne $10000 + 10J$ alors que la ligne en cours d'écriture est $10000 + 10(J - 1)$: l'affichage est décalé de dix.

N.d.R. Vérifié dans l'émulateur (vraie ROM) : les lignes gabarits doivent être tapées **sans espace** après `DATA` : `10000 DATA...,...,....` Après `RESTORE`, le pointeur `0328` désigne l'octet qui précède la ligne : le `+6` de la ligne 40 tombe donc juste après le jeton `DATA`. Le X-07 conserve l'espace tapée après `DATA` ; avec `10000 DATA ...`, les chiffres sont écrits un caractère trop tôt (`000.007.014...`, puis `12` au lieu de `112` à la ligne suivante). C'est d'ailleurs sans espace que LMDATA écrit les lignes données plus loin (`10000 DATA035,035,...`).

Exemple 1 : `RESTORE` calculé (fig. 17)

La routine du chapitre 1, implantée en `2000` avec des DATA produits par LMDATA.

```
1 REM Exemple de RESTORE CALCULE.
2 REM Cree par LMDATA
10 CLEAR50,&H1FFF:DEFINT A-Z
20 RESTORE10000
30 FORI=0TO17:READA:POKE&H2000+I,A:NEXT
40 INPUT"N= (1..3)";N
50 IFN<1 OR N>3 THEN 40
60 J=N*100
70 Z=USR(&H2000,J)
80 READA$:PRINTA$:GOTO40
100 DATA"lecture DATA 100"
200 DATA"lecture DATA 200"
300 DATA"lecture DATA 300"
999 END
10000 DATA035,035,094,035,086,205,013,243,096,105,210,056,246,043,034,040
10010 DATA003,201
```

Exemple 2 : logo CANON (fig. 18)

Programme BASIC qui dessine le logo CANON sur l'écran LCD par `PSET`. Chaque ligne d'écran (6 à 25) est décrite par des longueurs de segments, alternativement allumés et éteints, dont la somme vaut 79.

```
5 REM EXEMPLE DE GRAPHISME
```

```
10 CLEAR200:DEFINTA-Z
20 X=23:CLS
30 FORL=6TO25
40 A1=1:F=1
50 READA:A1=A1+A:IFF=1THEN70
60 F=1:GOTO80
70 FORI=A1-ATOA1-1:PSET(X+I,L):NEXT:F=0
80 IFA1=<79THEN50
90 NEXT
100 END
110 DATA79,9,4,66,8,6,65,6,10,63
120 DATA5,12,7,4,9,3,3,2,7,7,6,3,3,2,6
130 DATA4,6,6,2,5,6,7,5,1,4,5,9,4,5,1,4,5
140 DATA4,5,6,4,2,9,4,13,3,2,3,6,1,13,4
150 DATA3,6,6,2,10,3,6,5,2,5,2,3,2,6,3,5,2,5,3
160 DATA3,5,6,1,12,4,5,5,2,5,1,4,3,6,2,5,2,5,3
170 DATA3,5,15,8,5,5,2,5,1,4,3,6,2,5,2,5,3
180 DATA3,5,14,10,4,5,2,5,1,5,3,5,2,5,2,5,3
190 DATA3,5,13,5,2,4,4,5,2,5,1,5,3,5,2,5,2,5,3
200 DATA3,6,11,5,4,4,3,5,2,5,1,6,3,4,2,5,2,5,3
210 DATA4,5,9,1,1,5,4,4,3,5,2,5,1,6,3,4,2,5,2,5,3
220 DATA4,6,6,2,2,6,2,6,2,5,2,5,2,6,2,3,3,5,2,5,3
230 DATA5,12,4,8,1,4,2,5,2,5,2,6,3,2,3,5,2,5,3
240 DATA6,10,6,6,2,5,1,5,2,5,3,9,4,5,2,5,3
250 DATA8,6,9,4,3,5,1,5,2,5,4,7,5,5,2,5,3,79,79
```

Les valeurs ont été relues sur le scan et contrôlées : chacune des 20 lignes d'écran fait bien 79 points.

Exemple 3 : copie rapide de l'écran sur la X-710 (fig. 19)

La routine (53 octets, relogeable) est implantée dans une chaîne. Elle parcourt l'image miroir du LCD (`0214`, 4 lignes de 20 caractères), supprime les espaces de fin de ligne et envoie chaque ligne à l'imprimante (`CEF7`), suivie de CR et LF (`CFB0`).

Adr.	Octets	Étiq.	Instruction	Commentaire
00	06 00		LD B,0	numéro de ligne
02	21 14 02		LD HL,$0214	image miroir de l'écran
05	C5	L1	PUSH BC	
06	E5		PUSH HL	
07	01 13 00		LD BC,$0013	dernier caractère de la ligne
0A	09		ADD HL,BC	
0B	7E	L2	LD A,(HL)	
0C	FE 20		CP " "	espace de fin ?
0E	20 04		JR NZ,L3	
10	2B		DEC HL	
11	0D		DEC C	
12	20 F7		JR NZ,L2	
14	0C	L3	INC C	C = nombre de caractères utiles
15	E1		POP HL	
16	E5		PUSH HL	
17	7E	L4	LD A,(HL)	
18	04		INC B	
19	23		INC HL	
1A	E5		PUSH HL	
1B	C5		PUSH BC	
1C	CD F7 CE		CALL $CEF7	impression du caractère
1F	C1		POP BC	
20	E1		POP HL	
21	78		LD A,B	
22	B9		CP C	
23	20 F2		JR NZ,L4	
25	CD B0 CF		CALL $CFB0	CR + LF

Adr.	Octets	Étiq.	Instruction	Commentaire
28	E1		POP HL	
29	01 14 00		LD BC,$0014	ligne suivante
2C	09		ADD HL,BC	
2D	C1		POP BC	
2E	04		INC B	
2F	78		LD A,B	
30	FE 04		CP 4	4 lignes ?
32	20 D1		JR NZ,L1	
34	C9		RET	

```
1 REM Copie ecran
2 REM Methode de la chaine
3 REM DATA crees par LMDATA.
10 CLEAR 100:DEFINT A-Z
20 CH$=STRING$(53,"#")
30 GOSUB 300
40 FORI=0TO52:READA:POKEAD+I,A:NEXT
100 PRINT" Copie rapide de l'"
110 PRINT"      ecran sur"
120 PRINT" l'imprimante X-710"
130 PRINT"      +++";
200 GOSUB300:EXEC AD
210 END
300 AD=VARPTR(CH$):AD=PEEK(AD+1)+256*PEEK(AD+2):RETURN
10000 DATA006,000,033,020,002,197,229,001,019,000,009,126,254,032,032,004
10010 DATA043,013,032,247,012,225,229,126,004,035,229,197,205,247,206,193
10020 DATA225,120,185,032,242,205,176,207,225,001,020,000,009,193,004,120
10030 DATA254,004,032,209,201
```

9 LOGOGÉNÈSE (p. 89-90)

But. Créer des mots savants au hasard en associant un préfixe grec ou latin (« HYDRO », « PHILO »…) et un suffixe (« LOGIE », « PHOBE »…). Exemples possibles : « CHRONOPHAGE », « XYLOPHOBE ».

Principe. Le programme BASIC (1819 octets) tire une ligne de préfixes (100 à 132) et une ligne de suffixes (200 à 213). Il s'y place grâce au `RESTORE` calculé du chapitre 1, puis lit un mot au hasard (1 à 4) dans chaque ligne. La routine est logée dans la chaîne `RM$` ; son adresse est recalculée avant chaque appel (ligne 90), car la zone des chaînes se déplace.

```
1 'ATTENTION NE PAS RENUMEROTER
10 DEFINTA-Z:CLS:PRINT"Logogenese"
20 RM$=STRING$(18,0):GOSUB90
30 FORI=0TO17:READB$:POKEAD+I,VAL("&H"+B$):NEXT:Z=RND(0)
40 DATA23,23,5E,23,56,CD,0D,F3,60,69,D2,38,F6,2B,22,28,03,C9
45 J=INT(RND(1)*33)+100:GOSUB90
50 Z=USR(AD,J):N=INT(RND(1)*4+1)
55 FORI=1TON:READD$:NEXT
60 K=INT(RND(1)*14)+200:GOSUB90
65 Z=USR(AD,K):N=INT(RND(1)*4+1)
70 FORI=1TON:READF$:NEXT
75 PRINTD$;F$:IFINKEY$=""THEN45
80 IFINKEY$=""THEN80ELSE45
90 AD=VARPTR(RM$):AD=PEEK(AD+1)+256*PEEK(AD+2):RETURN
95 END
```

Ligne	Préfixes	Ligne	Préfixes / suffixes
100	AERO,AGRO,AMBI,ANDRO	117	MORPHO,MYTHO,NECRO,NEO
101	ANTI,ASTRO,BIBLIO,BIO	118	NOSO,NUCLEO,OCTO,OLEO
102	CACO,CALLI,CLEPTO,CHIMIO	119	OMNI,ORTHO,PALEO,PAN
103	CHRONO,CINE,COPRO,COSMO	120	PARA,PAPYRO,PATHO,PEDO
104	CRYO,CRYPTO,DECI,DECA	121	PENTA,PERI,PETRO,PHAGO
105	HEMI,DERMO,DEXTRO,DODECA	122	PHALLO,PHANERO,PHILO,PHYTO
106	DROMO,DYNAMO,DYS,ECTO	123	PHOBO,PHONO,PHOTO,PHRENO
107	ELECTRO,EMBRYO,ENTOMO	124	PHYSIO,POLY,MONO,HEMI
108	ERGO,EROTICO,GASTRO,GEO	125	PORNO,POST,PROTO,PSEUDO
109	GLOSSO,GONIO,GONO,GYMNO	126	PSYCHO,RADIO,RETRO,RHINO
110	HECTO,HELIO,HEMATO,HETERO	127	SADO,STEREO,SCLERO,SEMI
111	HIERO,HIPPO,HOLO,HOMEO	128	SCHIZO,SIMILI,SPELEO,STENO
112	HOMO,HYDRO,HYPER,HYPO	129	SUB,SUPRA,SUPER,TACHY
113	ICONO,IDEO,INFRA,ISO	130	TECHNO,TELE,TELEO,THEO
114	INTRA,LATERO,LIPO,LOGO	131	THERMO,TRIBO,ULTRA,VIDEO
115	LOXO,MACRO,MEGALO,METEO	132	XENO,XYLO,ZOO,MINI
116	META,MICRO,MISO,MNEMO		
200	ALGIE,CARDE,CINESE,CEPHALE	207	MATIQUE,MEGALIE,METRIE,MNESIE
201	CLASTE,COSMOS,CYCLE,DACTYLE	208	MORPHE,NAUTE,NEVROSE,NOMIE
202	DERME,DIDACTE,DOXE,DROME	209	PATHE,PHAGE,PHANIE,PHILE
203	DYNE,FUGE,GAME,GASTRE	210	PHOBE,PHONE,PNEE,PYGE
204	GLOTTE,GENESE,GENE,GRADE	211	RRHEE,SCAPHE,SCOPE,SCOPIE
205	GYNE,LATERE,LATRE,LOGIE	212	STASE,STENIE,THEISME,TONIQUE
206	LOGUE,MANCIE,MANE,MANIE	213	TROPE,TYPE,VORE,TIQUE

Chaque ligne commence par `DATA` (par exemple `100 DATA AERO,AGRO,AMBI,ANDRO`).

N.d.R. La ligne 107 n'a que trois mots : un tirage de 4 lit le premier mot de la ligne 108. C'est sans conséquence. La ligne 116 est peu lisible sur le scan (« JICRO, MKsO ») ; « MICRO, MISO » est la lecture probable.

10 LLIST (p. 91-95)

But. Améliorer la présentation des listings BASIC à l'imprimante : titre, numérotation des pages, saut de page, alignement des lignes et formatage des numéros. La figure 23 montre le résultat sur le logo CANON (« LISTING LOGO CANON – Page : 1 »).

Utilisation.

1. Vérifier par `DIR` et `FSET` qu'il reste au moins 600 octets libres dans la zone des fichiers RAM : la partie active s'installe au fond de la mémoire.
2. Charger les codes (`0800`–`0A29`).
3. Régler la longueur des pages (octet `0950`) et des lignes (octet `093C`, qui contient `27`). Mettre `26` évite des sauts de ligne imprévus avec une ligne de 39 caractères.
4. `EXEC &H800` : le relogeur copie LLIST au fond de la mémoire, affiche « NOTEZ AD= ...» puis se détruit. Il faut noter cette adresse (décimale, variable selon la version du X-07) : LLIST se lance ensuite par `EXEC adresse`.

LLIST analyse le programme comme REFBAS et EXABAS (voir §15). Il utilise la routine ROM `FEFE`, qui recopie dans le tampon d'entrée (`00D5`) la chaîne pointée par HL (terminée par 0). Longueur : 554 octets.

Le relogeur

Ce petit programme peut servir à reloger n'importe quelle routine. Il utilise une table des adresses à corriger, construite par l'assembleur (qui doit savoir calculer sur les étiquettes) ou à la main. Il ne faut en oublier aucune, et la table est à refaire à chaque modification du logiciel.

Étiq.	**Instruction**	**Commentaire**
`DEB`	`LD HL,($212)` `DEC HL`	RAMEND : fond de la mémoire

Étiq.	Instruction	Commentaire
	LD BC,fin-début	longueur à déplacer
	XOR A	CY = 0
	SBC HL,BC	HL = nouvelle adresse de début
	PUSH HL	(3 fois)
	CALL $CE9E	efface l'écran
	LD HL,MSG1	« NOTEZ AD = »
	CALL $FEF7	
	POP HL	
	CALL $BB98	affichage décimal de l'adresse
	LD HL,MSG2	« Pressez une touche »
	CALL $FEF7	
KEY	XOR A	
	CALL $C90A	attente d'une touche
	JR Z,KEY	
	POP HL	début après transfert
	LD DE,début	
	XOR A	
	SBC HL,DE	HL = décalage
	PUSH HL	
	POP BC	BC = décalage
	LD IX,table	adresses à modifier
DEB0	LD L,(IX+0)	
	LD H,(IX+1)	HL = adresse à modifier
	LD E,(HL)	
	INC HL	
	LD D,(HL)	DE = contenu
	DEC HL	
	PUSH HL	
	EX DE,HL	
	ADD HL,BC	contenu + décalage
	EX DE,HL	
	POP HL	
	LD (HL),E	modification
	INC HL	
	LD (HL),D	
	INC IX	entrée suivante
	INC IX	
	LD A,(IX+0)	
	CP 0	fin de la table ?
	JR NZ,DEB0	
	POP DE	DE = début (mémoire haute)
	LD HL,début	HL = début (mémoire basse)
	LD BC,fin-début	
	LDIR	transfert du programme modifié
	LD HL,DEB	
	LD A,$C3	écrit C3 C3 C3 sur DEB :
	LD B,3	le relogeur devient inutilisable
DEB1	LD (HL),A	
	INC HL	
	DJNZ DEB1	
	JP $C3C3	retour au BASIC
MSG1	DB 'NOTEZ AD =',0	
MSG2	DB 'Pressez ...',0	
TABLE	DW A1+1, ...	une entrée par adresse absolue
	DB 0	fin de la table

N.d.R. La fin de la table est testée sur l'octet de poids faible seulement : une adresse à corriger dont l'octet faible vaut `00` arrêterait la relocation. Le vidage de LLIST (fig. 21, `0800`–`0A29`) n'est pas recopié ici.

11 LABYRINTHE 3D (p. 96-100)

But. Sortir le plus vite possible d'un labyrinthe. Deux vues sont possibles : de « haut » (le dédale entier) ou de « bas » (vue en perspective de l'endroit où l'on se trouve). Une boussole guide le joueur et un score (le temps) s'affiche : le plus bas est le meilleur.

Utilisation. `EXEC &H800`, puis une touche après la présentation. Commandes :

N, W, E, S	tourner vers le nord, l'ouest, l'est, le sud	A	avancer
L	vue d'ensemble (coûte 20 points)	R	repartir du départ
B	retour au BASIC		

Le labyrinthe est créé automatiquement ; l'entrée est d'un côté, la sortie à l'opposé.

Algorithme de création du dédale (exploration en profondeur avec retour arrière) :

1. créer un tableau de cellules vides ;
2. choisir une origine proche du centre ;
3. tester les quatre cellules voisines pour savoir lesquelles ont été visitées ;
4. si toutes l'ont été, revenir en arrière jusqu'à une cellule visitée voisine d'une cellule non visitée ;
5. choisir au hasard une voisine non visitée ;
6. y aller en ouvrant une porte dans les deux cellules, et marquer la nouvelle cellule ;
7. tant que toutes les cellules n'ont pas été testées, revenir en 3 ; sinon créer une entrée et une sortie.

Longueur : 2066 octets, de `0800` à `1012` (vidage fig. 24). Pour les chargeurs (fig. 25) : `CLEAR50,&H7F0`, `D=&H800`, `F=&H1012`, nom de fichier « PLABIR ».

N.d.R. Le texte annonce 20 points pour la vue d'ensemble ; le message stocké dans le programme semble indiquer 22 points.

12 Le SOLITAIRE (p. 101-106)

Le jeu. Le solitaire se joue sur une planchette de 33, 37 ou 41 trous ; le logiciel utilise la version anglaise à 33 trous. Toutes les fiches sont en place sauf une. Une fiche saute horizontalement ou verticalement par-dessus une fiche voisine pour tomber dans un trou libre, et la fiche sautée est retirée. La partie s'arrête quand plus aucun coup n'est possible ; l'idéal est de ne garder qu'une fiche. Il n'y a aucun hasard.

```
          1   2   3
          4   5   6
  7   8   9  10  11  12  13
 14  15  16  17  18  19  20
 21  22  23  24  25  26  27
         28  29  30
         31  32  33
```

Utilisation. `EXEC &H800`.

— **Option 1, recherche par le X-07.** On choisit le nombre de fiches de la solution finale et l'un des 22 ordres d'examen des directions. Il n'existe pas d'algorithme simple : le programme essaie systématiquement les coups et revient en arrière niveau par niveau en cas d'impasse. L'ordre d'examen joue donc beaucoup sur la durée : avec 3 fiches ou plus, quelques secondes ; avec 2 fiches, de 5 minutes à une heure ; avec l'ordre 9, une solution classique en moins de 10 minutes.

— **Option 2, partie du joueur.** Le curseur se déplace avec les flèches. « D » marque la case de départ, « A » la case d'arrivée et joue le coup, « - » annule le coup précédent, « Q » arrête quand la partie est bloquée, CTRL Q revient au BASIC.

L'affichage de la profondeur d'analyse ralentit beaucoup la recherche ; on le supprime en mettant à 0 les octets `0989`, `098A` et `098B` par `POKE`.

Longueur : 4609 octets, de `0800` à `11D5` (vidage fig. 26). Chargeurs (fig. 27) : `D=&H800`, `F=&H11D5`, nom « SOL ».

13 Les PENTOMINOS (p. 107-113)

Le puzzle. Les 12 pentominos sont les 12 façons d'assembler cinq carrés par leurs côtés. On les désigne par une lettre de forme voisine : F, I, L, N, P, S, T, U, V, W, X, Y. Le casse-tête est connu depuis 1907 (*Canterbury Puzzles*). Nombre de façons de remplir un rectangle avec les 12 pièces :

Rectangle	3×20	4×15	5×12	6×10
Solutions	2	368	1010	2339

Utilisation. `EXEC &H800`, une touche après chaque affichage, puis choix du rectangle et de l'ordre d'examen des pièces. Le programme cherche par essais et erreurs ; l'ordre conditionne la durée et la solution trouvée. Le résultat peut être imprimé sur la X-710. CTRL Q revient au BASIC. L'ordre « TFILUVNWPXSY » donne une solution rapide pour les trois rectangles.

Longueur : 3489 octets, de `0800` à `159A` (vidage fig. 28). Chargeurs (fig. 29) : `D=&H800`, `F=&H159A`, nom « PENTA ».

14 AUTONUM (p. 114-116)

But. Ajouter au X-07 la numérotation automatique des lignes BASIC (fonction AUTO).

Installation. Choisir l'adresse de fin de la routine selon la mémoire du X-07 et la protéger par `FSET` (215 octets au moins). `RUN` demande cette adresse (en hexadécimal), affiche l'adresse de début (à noter) et lance la routine. Le chargeur peut ensuite être effacé par `NEW`.

Utilisation.

- CTRL N : un « ? » s'affiche ; taper le premier numéro et le pas, par exemple `100,10` : les lignes seront numérotées 100, 110, 120…
- CTRL Z : active ou suspend la numérotation.
- CTRL Q : désactive AUTONUM. Pour le relancer : `EXEC AD`, avec l'adresse affichée à l'installation.

Principe. AUTONUM détourne l'interruption du clavier (INT A, vecteur en `003D`). À chaque RETURN, la ligne est d'abord traitée normalement ; si AUTONUM est actif, le numéro suivant est ensuite écrit dans le tampon du clavier. Les cinq octets de données sont rangés dans le tampon de CSAVE/CLOAD : `0300` (indicateur actif/inactif), `0301`–`0302` (numéro courant), `0303`–`0304` (pas).

La routine est relogeable : elle doit trouver sa propre adresse pour l'écrire dans le vecteur. L'astuce consiste à écrire en `0300` la séquence `POP DE / PUSH DE / RET`, puis à l'appeler. Au retour, DE contient l'adresse de l'instruction qui suit le `CALL`.

Adr.	Octets	Étiq.	Instruction	Commentaire
00	21 D1 D5		LD HL,$D5D1	D1 = POP DE, D5 = PUSH DE
03	22 00 03		LD ($0300),HL	
06	3E C9		LD A,$C9	RET
08	32 02 03		LD ($0302),A	
0B	CD 00 03		CALL $0300	empile l'adresse XXX
0E	21 0E 00	XXX	LD HL,14	au retour, DE = XXX
11	19		ADD HL,DE	HL = AUTO (14 octets plus loin)
12	F3		DI	
13	22 3D 00		LD ($003D),HL	nouveau vecteur de INT A
16	FB		EI	
17	AF		XOR A	
18	32 00 03		LD ($0300),A	AUTONUM inactif au départ
1B	C9		RET	
1C	D9 08	AUTO	EXX ; EX AF,AF'	traitement de l'interruption
1E	DB F2 ...		IN A,($F2)...	tri des sources (comme la ROM)
33	DB F1		IN A,($F1)	touche reçue du T6834
35	FE 0D		CP $0D	RETURN : numéro suivant
84	FE 1A		CP $1A	CTRL Z : bascule

Adr.	Octets	Étiq.	Instruction	Commentaire
A3	FE 0E		CP $0E	CTRL N : saisie du numéro et du pas
D0	FE 11		CP $11	CTRL Q
D2	C2 80 C8		JP NZ,$C880	autre touche : traitement normal
D5	C3 C3 C3		JP $C3C3	réinitialisation (vecteur d'origine)

N.d.R. Le texte du livre écrit `LD ($300),(HL)` (instruction inexistante) : c'est `LD ($300),HL`. Il parle aussi de « cinq octets de mémoire morte » ; il s'agit de RAM. La routine fait 216 octets : le chargeur écrit de `D=F-215` à `F` inclus.

```
1 REM fonction AUTONUM pour X-07
10 CLEAR50,&HFFFF:DEFINTA-Z:CLS
20 INPUT"Adr. fin ";A$:F=VAL("&H"+A$)
30 D=F-215:PRINT"Adr. deb = ";HEX$(D)
40 FORI=DTOF:READA:POKEI,A:NEXT
50 EXECD:END
10000 DATA033,209,213,034,000,003,062,201,050,002,003,205,000,003,033,014
10010 DATA000,025,243,034,061,000,251,175,050,000,003,201,217,008,219,242
10020 DATA230,001,202,193,200,219,240,230,192,040,008,230,128,202,053,200
10030 DATA195,012,200,219,241,254,013,032,075,095,175,205,098,194,175,205
10040 DATA170,194,058,000,003,183,202,189,200,062,001,211,245,217,008,197
10050 DATA213,229,245,237,091,001,003,042,003,003,025,034,001,003,235,205
10060 DATA156,187,035,205,110,213,205,004,215,205,059,202,010,197,213,095
10070 DATA175,205,098,194,175,205,170,194,209,193,021,003,032,238,241,225
10080 DATA209,193,251,201,254,026,032,027,062,001,211,245,217,008,197,213
10090 DATA229,245,062,013,239,062,010,239,033,000,003,175,182,047,119,032
10100 DATA221,024,176,254,014,032,041,062,001,211,245,217,008,197,213,229
10110 DATA245,175,050,000,003,205,242,235,215,205,204,255,237,083,001,003
10120 DATA207,044,205,204,255,237,083,003,003,175,061,050,000,003,024,131
10130 DATA254,017,194,128,200,195,195,195
```

Ces lignes sont regroupées par 16 valeurs au lieu de la mise en page du livre ; les valeurs sont celles du livre, et le désassemblage du programme complet est cohérent.

15 REFBAS (p. 117-120)

But. Obtenir la liste triée des variables d'un programme BASIC avec les lignes où elles apparaissent, ou la table des références croisées des numéros de ligne. Utile pour analyser, renommer, réorganiser ou documenter un long programme.

Utilisation. Avant de charger REFBAS : `FSET 1024` (`FSET 4096` si la carte moniteur est en place), pour le protéger en fin de mémoire (version 16 Ko). Lancement : `EXEC &H3C10`.

- Option V : table triée des variables et des lignes où elles sont utilisées.
- Option S : références croisées pour GOTO, GOSUB, THEN, ELSE, RESUME, RUN et RESTORE. Chaque ligne cible est suivie des lignes qui y renvoient.
- Option B : retour au BASIC (CTRL Q fonctionne à tout moment ; BREAK pendant l'impression).

REFBAS construit, à la suite du programme analysé, une table de 6 octets par variable ou référence. Quand elle atteint la pile, il annonce « Mémoire insuffisante » : il faut alors analyser le programme par morceaux. La longueur des lignes imprimées (40 pour la X-710) se règle à l'adresse `3EDB`.

Exemple (sur le chargeur de la fig. 32).

```
Liste des variables
D       30 60 110 140 160
F       30 60 110 140 160
I       60 70 150 160 160
N$      30 50 120 140
T$      130
```

Parcours d'un programme BASIC. Comme tous les BASIC Microsoft, une ligne se compose de 2 octets de chaînage (adresse de la ligne suivante), 2 octets de numéro, du code (mots-clés sur un octet, le reste en ASCII) et d'un octet 0. Le début du programme est pointé par TXTTAB (`00B2`), normalement `0553`. REFBAS, EXABAS et LLIST utilisent la même boucle :

Étiq.	Instruction	Commentaire
	`LD IX,(TXTTAB)`	
`BOUCLE`	`XOR A`	
	`CP (IX+1)`	chaînage nul : fin du programme
	`JP Z,fin`	
	`PUSH IX`	début de la ligne
	`LD L,(IX+0)`	
	`LD H,(IX+1)`	HL = ligne suivante
	`LD E,(IX+2)`	
	`LD D,(IX+3)`	DE = numéro de ligne
	`PUSH HL`	
	`POP IX`	IX = ligne suivante
	`POP HL`	
	`LD BC,4`	
	`ADD HL,BC`	HL = premier code de la ligne
	`...`	traitement
	`JP BOUCLE`	

Longueur : 985 octets, de `3C10` à `3FE9` (vidage fig. 31). Chargeurs : `CLEAR50,&H9FF`, `D=&H3C10`, `F=&H3FE9`, nom « REFBAS ».

16 EXABAS (p. 121-124)

But. Compléter REFBAS par une recherche d'occurrences et un « chercher-remplacer » (fonctions REF et SR des grands systèmes).

Utilisation. `FSET 1024` (ou 4096 avec la carte moniteur), chargement comme REFBAS, puis `EXEC &H3C10`. Menu :

V	recherche d'une variable	c	remplacement d'une chaîne
C	recherche d'une chaîne (hors mots-clés)	t	remplacement d'un mot-clé
T	recherche d'un mot-clé	B	retour au BASIC

EXABAS affiche les numéros des lignes trouvées ou modifiées. Il marque une pause quand l'écran est plein ; la touche « I » envoie alors l'écran à la X-710. Un carreau signale la fin de l'analyse ; CTRL Q revient au BASIC. Faites une copie du programme avant un remplacement : la longueur des lignes change et EXABAS doit recalculer tous les chaînages qui suivent.

Exemple (sur le chargeur de la fig. 34).

```
Nom? N            30 50 120 140
Mot cl? PRINT     50 140
Mot cl? FOR       60 150 160
Mot cl? DEFSTR
Nom? T$           130
```

Longueur : 901 octets, de `3C10` à `3F95` (vidage fig. 33). Chargeurs : `CLEAR50,&H9FF`, `D=&H3C10`, `F=&H3F95`, nom « EXABAS ».

17 LE PIÈGE (p. 125-137)

But. Un jeu d'aventures, genre alors presque inexistant sur le X-07 : explorer une grande maison. Il faut visiter toutes les salles pour atteindre la sortie.

Utilisation. Deux versions, LCD et vidéo (X-720). Faire un `FSET` avant le chargement : la mémoire de `0800` à `1900` est utilisée.

Lancement	Effet
`EXEC &H800`	début du jeu
`EXEC &H815`	reprise à l'endroit quitté par « P »
`EXEC &H838`	reprise à l'endroit quitté par « Q »

L'écran affiche : ligne 1, le nom de la salle et les directions possibles ; ligne 2, les objets présents ; ligne 3, une description ; ligne 4, les commandes et les réponses.

Commandes. Une lettre : N, S, E, W pour se déplacer, I pour l'inventaire, Q pour quitter, P pour une pause (sauvegarde). Sinon, une phrase « verbe à l'infinitif + nom », par exemple `SAUTER COULOIR`. Seules les 4 premières lettres comptent (`SAUT COUL` suffit). Un verbe inconnu affiche « ? ? ? ? », un nom inconnu « ? ? ? ? ».

Version	Longueur	Implantation	Chargeurs
LCD (fig. 35–36)	4205 octets	`0800`–`186C`	
Vidéo (fig. 37–38)	4329 octets	`0800`–`18E8`	`CLEAR50,&H7F0`, `F=&H18E8`, nom « AVENT »

18 OTHELLO-REVERSI (p. 138-144)

Le jeu. Le Reversi a été inventé en Angleterre dans la seconde moitié du XIXe siècle, puis « réinventé » au Japon sous le nom d'Othello. Il se joue sur un plateau de 8×8 avec des pions blancs d'un côté et noirs de l'autre ; quatre pions sont posés au centre au départ. Chacun à son tour pose un pion de sa couleur de façon à encadrer au moins un pion adverse (en ligne, en colonne ou en diagonale) ; les pions encadrés sont retournés. Un retournement n'en provoque pas d'autre. Un joueur qui ne peut pas retourner de pion passe son tour. La partie s'arrête quand plus personne ne peut jouer ; le gagnant a le plus de pions. Sur le X-07, les deux camps sont représentés par des pions de formes différentes.

Utilisation. `EXEC &HE00` puis RETURN. Après la présentation, on choisit de jouer contre le X-07 ou contre un humain, le niveau (1 facile à 9 difficile) et ses initiales (4 lettres). Le damier s'affiche à gauche, le nombre de pions de chaque joueur à droite. Le point clignotant se déplace avec les quatre flèches ; RETURN joue le coup. « P » permet de passer si l'on est vraiment bloqué ; sinon, le X-07 affiche les coups possibles. CTRL Q quitte le jeu. Les auteurs jugent le programme très fort.

Longueur : 3073 octets, de `0800` à `1400` ; point d'entrée `0E00`.

19 Conclusion du livre (p. 145)

André Tonic rappelle que les deux livres sur le X-07 ont été écrits en six mois. Le service technique du Club C7 répond aux questions sur les deux ouvrages (courrier, téléphone, Minitel). Les Éditions Neptune promettent d'autres livres à bas prix. Le texte se termine sur « Bravo au X-07 ».

Annexes (p. 146-155)

20 Assembleurs et désassembleurs (annexe 1)

Logiciel	Appréciation des auteurs
Assembleur Logi'Stick	correct, mais étiquettes limitées, pas de directives (ORG, DEFM...), emploi compliqué
Assembleur Micro Systèmes	publié dans le n° 49 ; écrit en langage machine, tient sur une carte de 4 Ko, nombreuses pseudo-instructions ; préféré des auteurs, qui l'ont utilisé pour tous les listings du livre
Désassembleur-moniteur Logi'Stick	sur la même cassette que l'assembleur ; très bien fait ; 125 F au Club C7 (adhérents)
Carte Canon XP-140F	8 Ko de ROM et 4 Ko de RAM ; 10 instructions BASIC en plus (RENUM, MERGE, AUTO...) et un moniteur complet (points d'arrêt, désassemblage, entrée de codes) ; 500 à 550 F
Désassembleur-moniteur Micro Systèmes	publié dans le n° 42 ; complet, un peu moins fouillé que celui de Logi'Stick

21 Le Club C7 (annexe 2)

Association loi 1901 fondée en octobre 1984 par des professionnels de l'informatique et des étudiants. Elle propose une gazette bimestrielle, *Le Son du Canon* (environ 40 pages A4, envoyée avec une cassette des programmes), une programmathèque, une aide technique, une coopérative et des stages. Cotisation annuelle : 380 F (430 F pour l'étranger).

22 Les mystères du X-07 (annexe 3)

Présentation du premier livre : 160 pages en trois parties (assembleur Z80, mystères du X-07 avec près de 150 adresses, huit applications) et plus de 40 schémas.

23 La cassette (annexe 4)

Une cassette des Éditions Neptune (90 F, plus 15 F de port) contient les 15 programmes du livre : LMDATA, LOGO CANON, RESTORE, COPIE RAPIDE, LLIST, LOGOGÉNÈSE, LABYRINTHE, PENTOMINOS, SOLITAIRE, AUTONUM, REFBAS 16 Ko, EXABAS 16 Ko, LE PIÈGE (LCD et vidéo) et OTHELLO. Les programmes en assembleur y sont fournis en BASIC et en codes (B+C).

24 Bibliographie (annexe 5)

Éditeur	Titre
Sybex	*La programmation du Z-80*, R. Zaks
Eyrolles	*L'assembleur facile du Z-80*
P.S.I.	*Programmer en assembleur*, A. Pinaud
Canon	Manuel de la carte moniteur XP-140F
Club C7	Gazettes *Le Son du Canon* ; *Les mystères du X-07*
Tests	*L'Ordinateur Individuel*
S.P.E.	*Micro Systèmes*